Assignment 3 Report: Estimating the Impact and Value of Financial News

Nov 8, 2026

Team members: Daniel Diaz, Khoa Do (equal contribution)

Study period: 2012-2018

Submission date: November 8, 2026

1. Introduction

Headline text alone does not sufficiently predict market reaction. Using 178,959 company-day news events for S&P 500 companies in 2012–2018, the best IIS classifier (TF-IDF MLP) reached 57.72% test accuracy, below the 66.63% obtained by always predicting a neutral reaction (IIS = 0), and the best MIV regressor (DTM MLP) had $R^2 = 0.010$ (RMSE ≈ $1.64B). Individual words still shift the average reaction in the expected direction (for example, headlines containing "misses" average a –0.72% three-day CAR and "upgrade" +0.34%), but bag-of-words MLPs could not turn this into reliable event-level predictions.

This project measures the abnormal stock-price reaction around financial news for S&P 500 companies (2009–2020) and tests whether headline text can predict it. We measure the market reaction associated with each company-day of news, not proof that an article caused the whole move.

Pipeline:

- Collect news, the S&P 500 list and daily prices (0_Data_Prep.py).
- Run an event study to compute CAR, Z-score, IIS and MIV (1_calculate_impacts_values.py).
- Sample a 7-year period, split 80/20 by date, and vectorize headlines with DTM and TF-IDF (2_sampling_vectorize.py).
- Train PyTorch MLPs to classify IIS (3_classify.py) and regress MIV (4_regression.py).

2. Data collection

All three input files are built by 0_Data_Prep.py (final run: Nov 8, 2026). Row counts after each step are given below.

2.1 S&P 500 list (sp500.csv)

- Source: Wikipedia list of S&P 500 companies, retrieved Nov 8, 2026; 503 companies.
- Columns: ticker, company_name.
- Ticker format: Wikipedia uses BRK.B, Yahoo uses BRK-B; tickers in both sp500.csv and the news files were upper-cased and "." replaced with "-" so they match Yahoo.
- Limitation: using today's list for 2009–2020 introduces survivorship bias (companies that left the index are missing).

2.2 News (news.csv)

- Stacked analyst_ratings.csv and headlines.csv with pd.concat (same schema); dropped the id column, blank headlines and duplicate (date, symbol, headline, URL) items.
- Kept only S&P 500 tickers with downloaded price data (491 companies).
- Parsed timestamps, converted those with a UTC offset to US/Eastern, and mapped every item to its event date (Section 3.3); kept event dates from 2009-01-01 to 2020-12-31.
- Grouped by symbol and event date; joined headlines, URLs and publishers with the

separator ‖. The separator already appears in 48 of the ~3.25M source headlines, which is harmless because the tokenizer drops it.

- Event dates are assigned to each item before grouping, so an item published after 4:00 PM ET is grouped with the next trading day's news, not with earlier items from the same calendar date.

Step	Rows
analyst_ratings.csv (raw)	1,407,328
headlines.csv (raw)	1,845,559
After S&P 500 filter + 2009–2020 event dates	644,280 (325,507 + 318,773)
After concat + dropping blank/duplicate items	642,364
Company-day events (news.csv)	303,967

News covers 2009-04-29 to 2020-06-12 for 381 companies. Sample rows (URLs and publishers shortened):

date,symbol,headline,URL,publisher

2017-03-30,NRG,"Mid-Afternoon Market Update: Dow Rises Over 50 Points; Exfo Shares Tumble ‖ Mid-Day Market Update: Crude Oil Up Over 1.5%; Cartesian Shares Slide",https://www.benzinga.com/… ‖ …,Lisa Levin

2019-01-04,ALGN,"Morgan Stanley Maintains Equal-Weight on Align Technology, Lowers Price Target to $210 ‖ Align Technology (ALGN) Gains But Lags Market: What You Should Know ‖ …",https://www.benzinga.com/… ‖ …,Vick Meyer ‖ Zacks ‖ GuruFocus

2.3 Prices (historical_prices.csv)

- Source: Yahoo Finance via yfinance (auto_adjust=False), daily, 2008-01-01 to 2021-05-31 (last trading day 2021-05-28); 1,528,734 rows.
- Starting in 2008 gives the earliest 2009 events the 140 trading days of history needed for the estimation window; extending into 2021 gives late-2020 events their two following trading days.
- S&P 500 index (^GSPC) saved as ^SP500 (script 1 matches the symbol case-insensitively).
- Tickers with no usable price data in the period (mostly recent listings and spin-offs): 12 (CEG, FDXF, GEHC, GEV, HONA, HOOD, Q, RDDT, SNDK, SOLV, VLTO, VYLR); excluded.

2.4 Market capitalization

market_cap = current shares outstanding × close.

- Shares outstanding: today's value from Yahoo (fast_info["shares"], falling back to sharesOutstanding).
- Close: Yahoo's split-adjusted close, so the share count and price are on the same split basis.
- Assumption: share count is constant over time. Buybacks and issuance are ignored, so older market caps are approximate (understated for companies with large buybacks, e.g. AAPL ≈ $86.9B on 2008-06-30 in our data vs ≈ $145B actual). This equals the assignment's second formula, market_cap_current × close_t / close_current.
- ^sp500 market_cap left blank.

Sample rows (prices rounded):

date,symbol,open,high,low,close,adj_close,volume,market_cap

2015-06-01,AAPL,32.57,32.85,32.51,32.63,29.10,128451200,474257735178

2015-06-01,JPM,65.99,66.66,65.97,66.08,48.79,11899300,175482681217

2015-06-01,^SP500,2108.64,2119.15,2102.54,2111.73,2111.73,3011710000,

3. Event study methodology

1_calculate_impacts_values.py computes, for every stock and trading day t, the market reaction as if t were a news day, and saves it to historical_impacts_values.csv.

3.1 Returns and volatility

$R(i,t) = P(i,t)/P(i,t-1) - 1$

- P = adj_close. Market return R_m uses ^SP500.
- daily_volatility = standard deviation of the last 60 daily returns.

3.2 Market model and estimation window

$R(i,t) = \text{alpha} + \text{beta}\ R(m,t) + \text{epsilon}(i,t)$

- α and β fitted by OLS on trading days t−140 to t−21 (120 days, ending 20 days before t).
- No event-window data is used, avoiding look-ahead bias.

3.3 Event date

- News before 4:00 PM ET on a trading day → that day.
- News at or after 4:00 PM ET → next trading day.
- Weekend or holiday → next trading day.
- Timestamps with a UTC offset (~4% of analyst_ratings rows) are converted to US/Eastern first. Date-only timestamps (all of headlines.csv and ~96% of analyst_ratings.csv) have no time of day, so they are assigned to that date if it is a trading day, otherwise to the next trading day. Trading days come from the ^SP500 price series.

3.4 AR, CAR, Z-score, IIS, MIV

$AR(i,t) = R(i,t) - (\alpha + \beta \cdot R(m,t))$ $CAR(i,t) = AR(i,t) + AR(i,t+1) + AR(i,t+2)$

$Z_CAR = (CAR_event - \mu_CAR) / \sigma_CAR$ $MIV(i,t) = MC(i,t-1) \times CAR(i,t)$

- μ and σ: mean and standard deviation of the 118 overlapping 3-day CARs inside the 120-day estimation window, using the same α and β.
- MC = market cap on the day before the event.
- IIS: $Z \le -3 \rightarrow -3$; $-3 < Z \le -2 \rightarrow -2$; $-2 < Z \le -1 \rightarrow -1$; $-1 < Z < 1 \rightarrow 0$; $1 \le Z < 2 \rightarrow +1$; $2 \le Z < 3 \rightarrow +2$; $Z \ge 3 \rightarrow +3$.

3.5 Exclusions

- Rows without a full estimation window, gap or event window (missing prices): 68,406.
- 136,498 company-days (83 companies that began trading after January 2008) were excluded because a missing first daily return propagated through the rolling sums — a known bug in our script, not a property of the data.
- Remaining: 1,319,064 scored company-days for 408 companies, 2008-07-24 to 2021-05-26.

3.6 Worked example and output

ADP, 2016-11-02 (headline: “ADP Reports Q1 Adj. EPS $0.86 vs $0.76 Est., Sales $2.9B vs $2.92B Est. || Earnings Scheduled For November 2, 2016”): R = +3.52%, R_m = –0.65%, α = –0.000443, β = 1.048, AR = +4.25%, CAR = +3.97%, Z = 2.84, IIS = +2, MIV = $1.363B (MC_pre ≈ $34.312B).

IIS	Count	%
–3	15,459	1.17
–2	29,248	2.22
–1	133,389	10.11
0	963,421	73.04

+1	132,970	10.08
+2	29,493	2.24
+3	15,084	1.14

Sample rows (selected columns, rounded):

date,symbol,close,market_cap,daily_return,market_return,alpha,beta,AR,CAR,Z_CAR,IIS,MIV

2014-02-11,ADSK,52.92,1.106e+10,0.0122,0.0111,0.00217,1.200,-0.0032,0.0086,0.34,0,9.372e+07

2018-06-29,TMUS,59.75,6.329e+10,-0.0018,0.0008,-0.00103,0.832,-0.0014,-0.0080,-0.34,0,-5.067e+08

2019-10-11,CSGP,61.41,2.471e+10,0.0203,0.0109,0.00138,1.016,0.0078,-0.0264,-1.13,-1,-6.399e+08

4. Text preprocessing and sentiment terms

4.1 Preprocessing

Step	What it does	Why it matters
Remove separator	No explicit step: the ‖ separator contains no word characters, so the tokenizer drops it	Separator is not language
HTML / noise removal	BeautifulSoup strips tags, script/style/nav/footer/aside elements and ad-labelled elements; whitespace collapsed	Keeps only headline content
Lowercase + tokenize	Lowercased; contractions expanded (contractions.fix, “don't” → “do not”); scikit-learn token pattern \b\w\w+\b	Merges duplicate tokens
Remove punctuation and numbers	Punctuation removed by the token pattern; numbers with 2+ digits (e.g. price targets such as 210) kept as tokens; 1-character tokens dropped	Reduces sparse noise
Stop words	None removed (stop_words=None), so “not” and “no” are kept	Avoids deleting negations and financial words
Negation handling	None: “not” is kept as its own unigram but not joined to the next word	“not” flips meaning; left as an improvement
Vectorizer limits	max_features=5000, unigrams, min_df=1, max_df=1.0	Controls vocabulary size; the full vocabulary (~43,000 terms) did not fit in 16 GB RAM as dense vectors

4.2 Five sentiment terms

Term	Expected direction	Headlines containing it	Mean CAR	Mean IIS
upgrade	+	13,154	+0.337%	+0.089

downgrade	–	11,501	–0.502%	–0.156
beat	+	14,714	+0.360%	+0.103
miss	–	6,557	–0.720%	–0.225
cut	–	3,595	–0.533%	–0.159

Justification: computed over all 275,540 matched news events from 2009–2020 (overall mean CAR +0.024%, mean IIS ≈ 0); word forms such as upgrades/upgraded and misses/missed are included. **upgrade / downgrade** are analyst rating changes, which directly revise the market's estimate of value, and their reactions have opposite signs (+0.34% vs –0.50%). **beat / miss** describe earnings versus analyst expectations; the surprise, not the level, moves the price, and "miss" is the strongest negative term of the five (–0.72%). **cut** covers dividend, guidance and price-target cuts, all negative revisions (–0.53%). Related terms point the same way: "lowers" –1.06% and "raises" +0.56%.

Context caveat: "loss" alone is almost neutral (mean CAR –0.11% over 2,254 events; positive and negative IIS each ≈ 22%) because it appears in both bad and good news. For example, "Vertex Pharmaceuticals Narrows Loss" (VRTX, 2011-07-28) was followed by a +6.7% CAR (IIS +1); the 50 "narrow… loss" events averaged +1.68%.

5. Sampling and vectorization

2_sampling_vectorize.py selects a 7-year period, splits it by date and builds the four vectorized datasets.

- Seed: np.random.seed(42), start year drawn from 2009–2014 → 2012.
- Period: 2012-01-01 to 2018-12-31 (1,760 trading days).
- Split: chronological 80/20 by trading day (first 1,408 days for training). Train 2012-01-03 to 2017-08-07; test 2017-08-08 to 2018-12-31.
- News events mapped to event dates (Section 3.3), then matched to IIS and MIV in historical_impacts_values.csv.

Leakage controls:

- An event is kept only if all 3 CAR days fall inside its own period; 209 training events (2017-08-04 and 2017-08-07) whose window crossed into the test period, and 228 test events on the last two trading days of 2018, were dropped (437 of 179,396 matched events).
- Vectorizers (DTM and TF-IDF) fit on the training headlines only; test headlines transformed with the same vocabulary and IDF.

Dataset	Rows	Vocabulary size
news_dtm_train	131,470	5,000
news_dtm_test	47,489	same
news_tfidf_train	131,470	5,000
news_tfidf_test	47,489	same

Vectorizer settings: CountVectorizer(max_features=5000) and TfidfVectorizer(max_features=5000); otherwise defaults (min_df=1, max_df=1.0, ngram_range=(1, 1), lowercase, no stop words). Each news_vector is stored as a dense JSON list of 5,000 values.

6. IIS classification

TF-IDF reached 57.72% test accuracy versus 54.81% for DTM, but both are below the 66.63% majority-class baseline. Training accuracy was about 94.5% for both, so the models overfit the training headlines.

6.1 Model (3_classify.py)

Setting	Value	Why
Layers	5,000 → 128 → 64 → 7	Two narrowing hidden layers; 7 outputs = IIS −3 … +3
Activation	ReLU; softmax via CrossEntropyLoss	Standard choice for MLP classifiers
Dropout	0.1 after each hidden layer	Light regularization
Optimizer, learning rate	Adam, 1e−3	Robust default
Batch size, epochs	32, 60 (seed 42, shuffled batches)	Same for both vectorizers
Class weights	No	IIS is mostly 0

Identical settings for DTM and TF-IDF.

6.2 Results

Majority-class baseline (always predict 0): 66.63% test accuracy.

Company	Test events	DTM accuracy (%)	TF-IDF accuracy (%)
All test data	47,489	54.81	57.72
WDC	289	51.21	58.82
LUV	283	61.84	61.84
NFLX	283	54.42	59.72
KR	282	62.77	71.28
REGN	282	58.87	62.77
MDT	280	61.43	68.21
GILD	276	63.41	68.12
SLB	272	51.10	58.82
AXP	269	61.34	60.22
TXN	267	56.93	60.67

6.3 Discussion

TF-IDF beat DTM overall (+2.9 points) and for 8 of the 10 largest companies (tied for LUV, lower for AXP), likely because down-weighting words that appear in most headlines (“market”, “update”, “stocks”) leaves more company-specific signal. Neither model beats the 66.63% baseline: about two-thirds of events are neutral, and with no class weighting the networks memorize the training headlines (~94.5% training accuracy) instead of learning patterns that generalize. Per-company accuracy ranges from 51% (WDC, DTM) to 71% (KR, TF-IDF).

7. MIV regression

DTM had a slightly lower RMSE ($1.641B vs $1.647B) and higher R^2 (0.0100 vs 0.0024), but both R^2 values are close to zero: headline text explains about 1% of the variation in MIV.

7.1 Model (4_regression.py)

Setting	Value	Why
Layers	5,000 → 128 → 64 → 1 (dropout 0.1 after each hidden layer)	Same architecture as the classifier
Activation	ReLU; linear output	Unbounded dollar target
Loss	MSE	Matches RMSE evaluation
Target scaling	None: raw MIV in dollars	MIV spans several orders of magnitude; scaling is a recommended improvement

Optimizer, learning rate	Adam, 1e−3	Same as classifier
Batch size, epochs	32, 60 (seed 42)	Same for both vectorizers

7.2 Results

Company	Test events	DTM RMSE ($)	DTM R^2	TF-IDF RMSE ($)	TF-IDF R^2
All test data	47,489	$1.641B	0.0100	$1.647B	0.0024
WDC	289	$770.6M	0.0712	$787.1M	0.0309
LUV	283	$804.7M	–0.0148	$803.4M	–0.0115
NFLX	283	$4.839B	0.0141	$4.873B	–0.0000
KR	282	$593.7M	0.0110	$599.5M	–0.0084
REGN	282	$1.215B	–0.0074	$1.211B	–0.0003
MDT	280	$1.650B	0.0211	$1.665B	0.0034
GILD	276	$2.302B	0.0312	$2.329B	0.0085
SLB	272	$2.324B	–0.0046	$2.326B	–0.0069
AXP	269	$1.099B	0.0375	$1.117B	0.0057
D	267	$1.234B	–0.0122	$1.230B	–0.0052

7.3 Discussion

DTM was marginally better, but the difference ($6M of RMSE) is negligible. R^2 is near or below zero because MIV is extremely noisy (test median ≈ –$1.1M, standard deviation ≈ $1.65B, range –$38.8B to +$33.9B) and headlines explain little of the abnormal return. Because MIV = market cap × CAR, large companies dominate RMSE: NFLX has an RMSE of ≈ $4.8B, versus ≈ $0.6B for KR. Training on raw dollars with MSE also lets a few huge events drive the fit. Predicting CAR or Z (unit-free) and multiplying by market cap afterwards would likely be easier.

8. Analysis and reflection

8.1 Key findings

- Headline text has weak predictive power: the best IIS classifier (TF-IDF, 57.72%) is below the 66.63% always-neutral baseline, and MIV regression reaches $R^2 \approx 0.01$.
- TF-IDF outperformed DTM for classification (57.72% vs 54.81%); DTM was marginally better for regression (RMSE $1.641B vs $1.647B).
- Individual terms still carry signal on average: “misses” –0.72%, “downgrade” –0.50%, “upgrade” +0.34% mean three-day CAR, but the MLPs overfit (~94.5% training vs ~56% test accuracy).

8.2 Limitations

- Survivorship bias: today's S&P 500 list used for 2009–2020.
- Market cap uses today's share count; older values are approximate.
- Headlines only, no article text; several headlines combined per company-day, so the reaction can't be tied to one article.
- Other events in the 3-day window (earnings, macro news) are mixed into the CAR.
- Most timestamps are date-only, so the 4 PM rule only applies to the ~4% of items with a time; date-only items published after the close may be assigned one trading day too early.
- IIS is heavily imbalanced toward 0.
- Vocabulary capped at 5,000 terms because the required dense news_vector format did not fit in our computer of 16 GB of RAM with the full vocabulary.
- 136,498 company-days (83 companies that began trading after January 2008) were excluded because a missing first daily return propagated through the rolling sums — a known bug in our script, not a property of the data.

8.3 Challenges and lessons learned

- Environment: the scripts use Python 3.10+ syntax (str | None), so the macOS system Python 3.9 failed; a Python 3.12 virtual environment fixed it.
- Memory and runtime: with the full ~43,000-term vocabulary the dense vector CSVs would have been ≈ 45 GB and the classifier needed > 22 GB RAM, so the vocabulary was capped at 5,000 terms. Final runtimes: data prep ≈ 5 min, event study ≈ 1 min, vectorization ≈ 4 min, classification ≈ 34 min, regression ≈ 11 min.
- Event dates: an early version used the UTC calendar date and dropped weekend/holiday news; applying the 4 PM ET / next-trading-day rules increased the train + test events from 173,566 to 178,959.
- Timestamp formats are mixed (date-only vs. UTC-offset), so they had to be parsed separately.
- Lesson: compare against a majority-class baseline; raw accuracy alone overstated model quality.

8.4 Improvements

- Contextual embeddings (e.g. FinBERT) instead of bag-of-words.
- Predict CAR or Z directly, then convert to MIV.
- Class weighting or resampling for IIS.
- Historical constituent lists and historical share counts.
- Full article text where licensing allows.
- Store vectors sparsely so the full vocabulary fits in memory; add bigrams and negation handling.
- Early stopping or stronger regularization to reduce overfitting.

Appendix: files and run order

Order	Script	Reads	Writes
0	0_Data_Prep.py	analyst_ratings.csv, headlines.csv, Wikipedia, Yahoo Finance	sp500.csv, news.csv, historical_prices.csv
1	1_calculate_impacts_values.py	historical_prices.csv	historical_impacts_values.csv
2	2_sampling_vectorize.py	news.csv, historical_impacts_values.csv	news_dtm_train/test.csv, news_tfidf_train/test.csv
3	3_classify.py	news_dtm_, *news_tfidf_*	Console output (accuracy, top-10 table)
4	4_regression.py	news_dtm_, *news_tfidf_*	Console output (RMSE, R^2, top-10 table)